

Chapter 6-1

Intermediate Representation (IR)

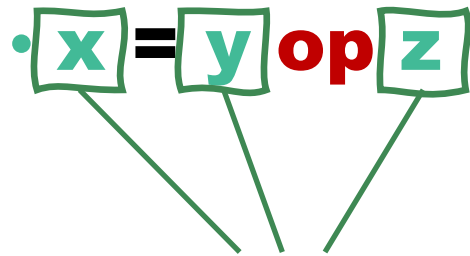
Intermediate Representation

- **Part I: Three-Address Code**
 - Definition & Implementations of Three-Address Code
- **Part II: Generation of Three-Address Code**
 - Syntax-Directed Translation
 - Generating IR for Variable Declaration/Expressions/Statements
 - Generating IR for Control Flows

PART I: Three-Address Code

Three-Address Code

- Provide a standard representation, not as flexible as the source
- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction



Variables or constant;

Variables are from the source code or created by the compiler

Three-Address Code

Instruction Type	Instruction Forms	Examples
<u>Computation and Assignment</u>		
Arithmetic Operation	$x = y \oplus z;$ $x = \ominus z;$	$x = y + z;$ $x = -z;$
Logic Operation	$x = y \oplus z;$ $x = \ominus z;$	$x = y \&\& z;$ $x = !z;$
Relational Operation	$x = y \oplus z;$	$x = y > z;$
Copy/Assignment	$x = y;$	-
<u>Control Flow</u>		
Unconditional Jump	goto L;	-
Conditional Jump	if x goto L else goto L';	=> if x goto L;
	if $x \oplus y$ goto L else goto L';	if $x > y$ goto L else goto L';
		=> if $x > y$ goto L;
		$\equiv z = x > y;$ if z goto L;

Three-Address Code

Instruction Type	Instruction Forms	Examples
<u>Memory Operations</u>		
Indexed Copy	$x = y[z]; \quad y[z] = x;$	$x = y[z]; \equiv t = y + z; x = *t;$ $y[z] = x; \equiv t = y + z; *t = x;$
Address-Taken	$x = \&y;$	-
Load Operation	$x = *y;$	-
Store Operation	$*x = y;$	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
<u>Memory Operations</u>		
Indexed Copy	<code>x = y[z]; y[z] = x;</code>	<code>x = y[z]; $\equiv$ t = y + z; x = *t;</code> <code>y[z] = x; $\equiv$ t = y + z; *t = x;</code>
Address-Taken	<code>x = &y;</code>	-
Load Operation	<code>x = *y;</code>	-
Store Operation	<code>*x = y;</code>	-
<u>Function Call and Return</u>		
Function Call	<code>param p₁;</code> <code>param p₂;</code> <code>...</code> <code>param p_n;</code> <code>x = call func n; call func n;</code>	<code>param 1;</code> <code>param 2;</code> <code>call foo 2;</code> <code>=> foo(1, 2)</code>
Function Return	<code>return; return x;</code>	-

Three-Address Code

- `do i = i + 1; while (a[i + 2] < v);`

```
L:  t1 = i + 1
    i = t1
    t2 = i + 2
    t3 = a [ t2 ]
    if t3 < v goto L
```

Symbolic Labels

```
100: t1 = i + 1
101: i = t1
102: t2 = i + 2
103: t3 = a [ t2 ]
104: if t3 < v goto 100
```

Numeric Labels

- Implementation methods: *quadruples*, *triples*, etc.

Quadruples

- $a = b * (-c) + b * (-c)$

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a   = t5
  
```

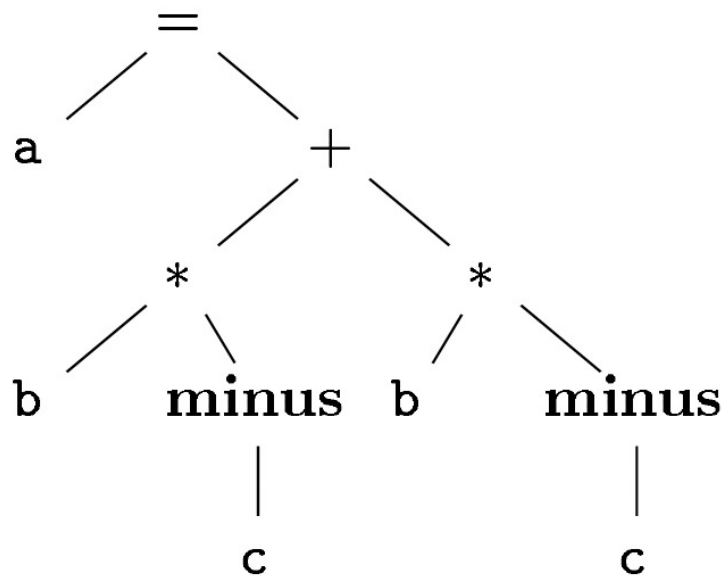
Three-Address Code

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

Quadruples

Triples

- $a = b * (-c) + b * (-c)$



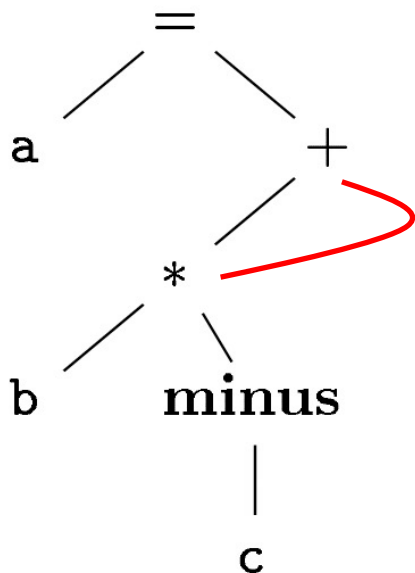
Syntax Tree

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples with Syntax DAG

- $a = b * (-c) + b * (-c)$



Syntax DAG

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(1)
5	=	a	(4)
	...		

Triples

Indirect Triples with Syntax DAG

- $a = b * (-c) + b * (-c)$

<i>instruction</i>			<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
35	(0)	→	0	minus	c
36	(1)	→	1	*	b (0)
37	(2)	→	2	minus	c
38	(3)	→	3	*	b (2)
39	(4)	→	4	+	(1) (1)
40	(5)	→	5	=	a (4)
...				...	

Indirect Triples

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(1)
5	=	a	(4)
...			

Triples

PART II-1: Syntax-Directed Translation

Syntax-Directed Definition (SDD)

- SDD = Context-Free Grammar + Attributes + Rules
- **Attributes** are associated with **symbols** in the grammar
- **Rules** are associated with **productions** in the grammar

Syntax-Directed Definition (SDD)

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \text{ n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Rules of computing
the result

Attributes

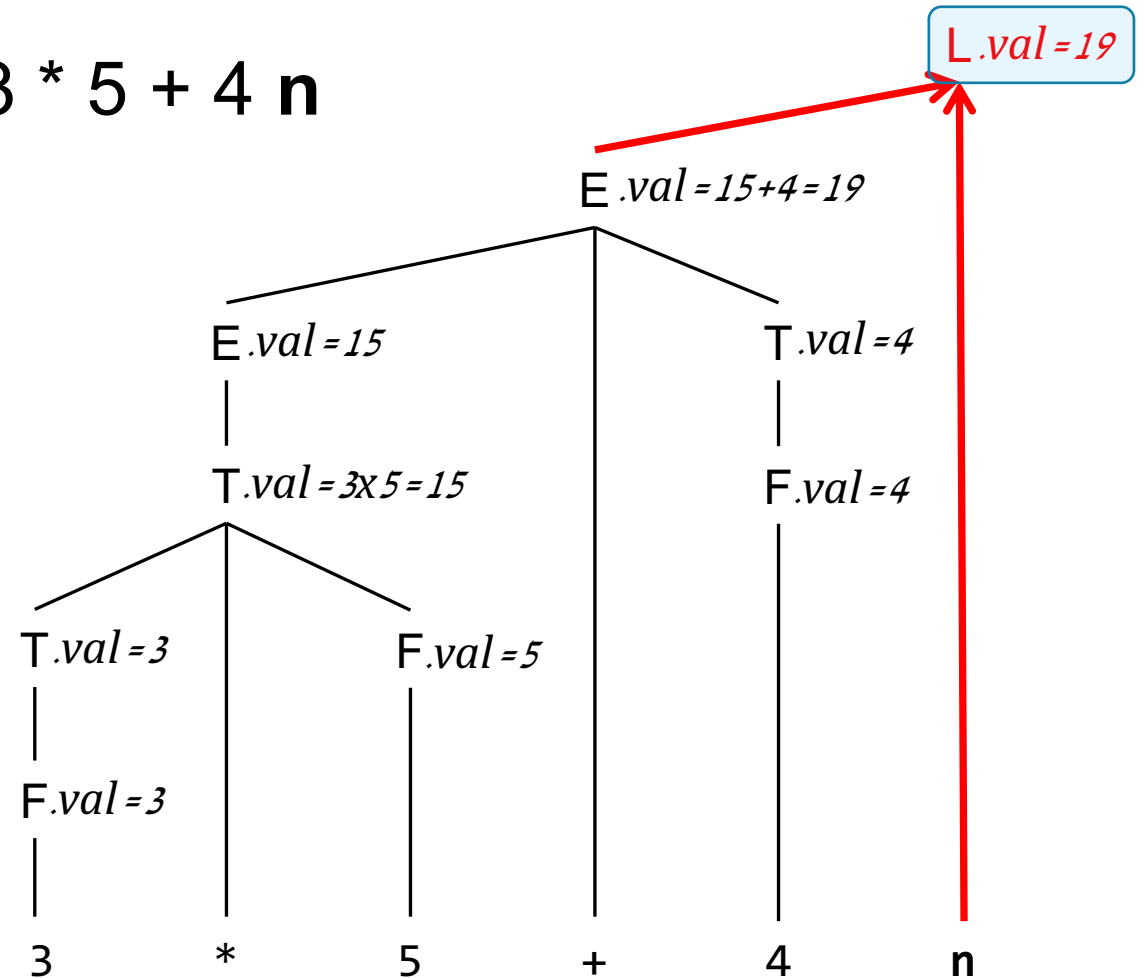
Attributes

* Grammar for arithmetic expressions

Understanding SDD via AST

- **Example:** (Annotated) AST for $3 * 5 + 4 n$

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



Understanding SDD via AST

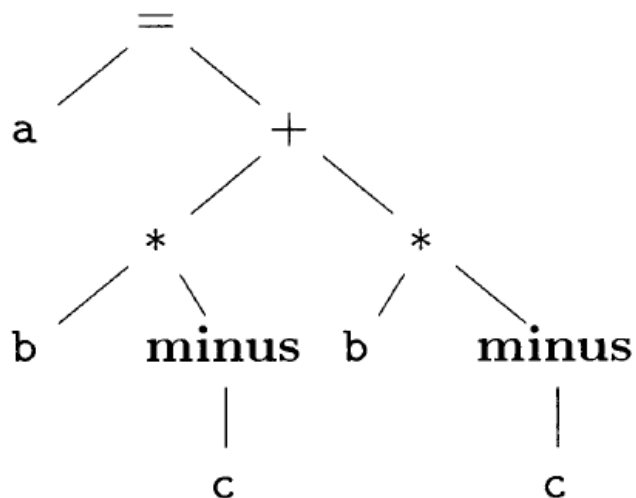
- SDD provides semantic rules that allow us to interpret a char string by traversing the syntax tree.

Syntax-Directed Translation (SDT)

- Using SDD to define a series of rules, which translate a string in a high-level language into another language.
- Particularly useful for IR generation.

Generation of Syntax Tree

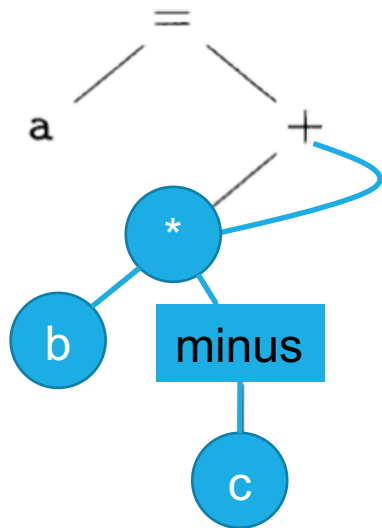
• $a = b * (-c) + b * (-c)$



Production	Rules
Assignment $\rightarrow$ variable = Exp	node(=).addChild(node(variable), node(Exp))
$\text{Exp}_1 \rightarrow \text{Exp}_2 + \text{Exp}_3$	node(+).addChild(node(Exp ₂), node(Exp ₃))
$\text{Exp}_1 \rightarrow \text{Exp}_2 * \text{Exp}_3$	node(*).addChild(node(Exp ₂), node(Exp ₃))
$\text{Exp}_1 \rightarrow (-\text{Exp}_2)$	node(minus).addChild(node(Exp ₂))
$\text{Exp} \rightarrow$ variable	node(variable)

Generation of Syntax DAG

• $a = b * (-c) + b * (-c)$



Production	Rules
Assignment $\rightarrow$ variable = Exp	<code>node(=).addChild(node(variable), node(Exp))</code>
$\text{Exp}_1 \rightarrow \text{Exp}_2 + \text{Exp}_3$	<code>node(+).addChild(node(Exp₂), node(Exp₃))</code>
$\text{Exp}_1 \rightarrow \text{Exp}_2 * \text{Exp}_3$	<code>node(*).addChild(node(Exp₂), node(Exp₃))</code>
$\text{Exp}_1 \rightarrow (-\text{Exp}_2)$	<code>node(minus).addChild(node(Exp₂))</code>
$\text{Exp} \rightarrow$ variable	<code>node(variable)</code>

PART II-2: Translating Variable Declarations

Variable Declaration

```
int main(){  
    int a; int b; int c;  
    scanf("%d%d", &a, &b);  
    c = a + b;  
    printf("%d", c);  
    return 0;  
}
```

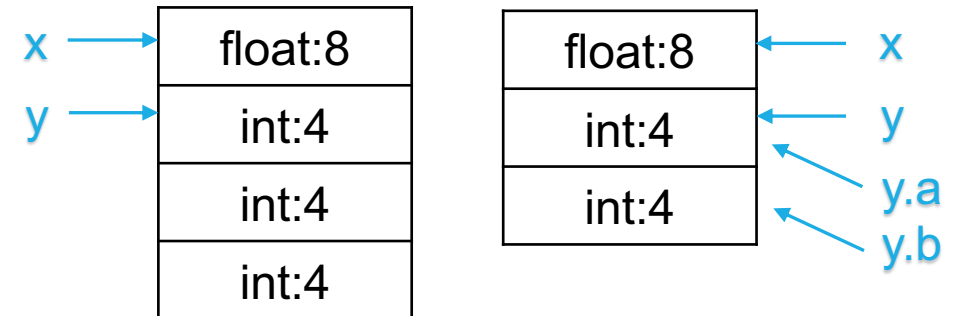
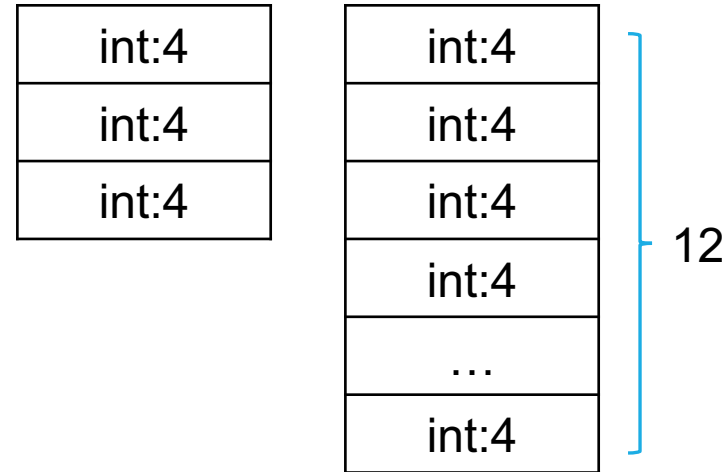
Declaration $\rightarrow$ TypeExpression Variable;

Generating IR for variable declaration:

- 1. Type of a variable!**
- 2. Memory layout!**

Type Expression

- Primary Types
 - int, char, float, ...
- Composite Types
 - type expression [number]
 - int[3]
 - int[3][4]
 - record {list of declarations}
 - record {float x; int[3] y;}
 - record {float x; record {int a; int b;} y}



Variable Declaration

$$\begin{aligned}
 D &\rightarrow T \text{ id } ; D \mid \epsilon \\
 T &\rightarrow B C \mid \text{record } '\{ ' D '\}' \\
 B &\rightarrow \text{int} \mid \text{float} \\
 C &\rightarrow \epsilon \mid [\text{num}] C
 \end{aligned}$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$\begin{array}{ll} T \rightarrow B & \{ t = B.type; w = B.width; \} \\ C & \{ T.type = C.type; T.width = C.width; \} \end{array}$$

$$B \rightarrow \text{int} \quad \{ B.type = \text{integer}; B.width = 4; \}$$

$$B \rightarrow \text{float} \quad \{ B.type = \text{float}; B.width = 8; \}$$

$$C \rightarrow \epsilon \quad \{ C.type = t; C.width = w; \}$$

$$C \rightarrow [\text{num}] C_1 \quad \{ C.type = \text{array}(\text{num.value}, C_1.type); \\ C.width = \text{num.value} \times C_1.width; \}$$

SDT for Variable Declarations

$$\{ \textit{offset} = 0; \}$$

$$D \rightarrow T \textbf{id} ; \quad \{ \textit{top.put}(\textbf{id.lexeme}, T.\textit{type}, \textit{offset});$$

$$\textit{offset} = \textit{offset} + T.\textit{width}; \}$$

$$D \rightarrow \overset{D_1}{\epsilon}$$

SDT for Record Type

$$\begin{aligned} T \rightarrow \text{record } \{'\{ ' & \{ \text{Env.push}(top); top = \text{new Env}(); \\ & \text{Stack.push}(\text{offset}); \text{offset} = 0; \} \\ D \}' & \{ T.type = \text{record}(top); T.width = \text{offset}; \\ & top = \text{Env.pop}(); \text{offset} = \text{Stack.pop}(); \} \end{aligned}$$

Read the Dragon book for details...

PART II-3: Translating Statements

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(\mathbf{id.lexeme}) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr '=' E_1.addr '+' E_2.addr)$
$E \rightarrow E_1 - E_2$	$E.addr = \mathbf{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr '=' 'minus' E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$E \rightarrow \mathbf{id}$	$E.addr = top.get(\mathbf{id.lexeme})$ $E.code = ''$

Type Checking

- Checking types for assignment
 - $S \rightarrow \text{id} = E$ { id.type vs. $E.type$ }
- Similar cases for function overloading
 - $E \rightarrow \text{id}(\text{ParamList})$ { compare id.type vs. ParamList.types }

PART II-4:

Translating Control Flows

Outline

- if-else statement; while statement
- break and continue
- Boolean expressions and short-circuit

Control Flow

***.code:** code generated by a nonterminal

S.next: next statement

B.true: true branch of a boolean expression

B.false: false branch of a boolean expression

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \ \ B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \ label(B_1.false) \ \ B_2.code$
$B \rightarrow B_1 \ \&\& \ B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \ label(B_1.true) \ \ B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \ \mathbf{rel} \ E_2$	$B.code = E_1.code \ \ E_2.code$ $\quad \ \ gen('if' \ E_1.addr \ \mathbf{rel.op} \ E_2.addr \ 'goto' \ B.true)$ $\quad \ \ gen('goto' \ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('goto' \ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('goto' \ B.false)$

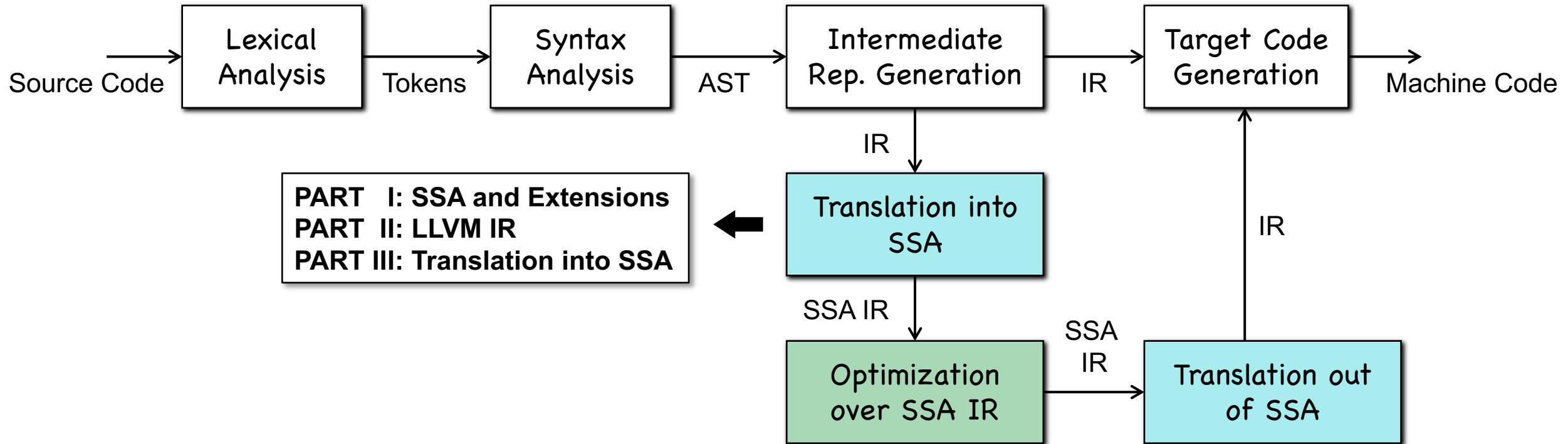
Others

- Optimizations:
 - Avoid unnecessary gotos
 - Backpatching
- Translation for:
 - Switch statement
 - Function / Function call
- ***Read the dragon book, Chapter 6.***

Chapter 6-2

Translation into SSA Form

SSA in Compilers



PART I: SSA and Extensions

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

(a) Three-address code. (b) Static single-assignment form.

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```



```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\phi$  (x1, x2);
y = x3 * a
```

Static Single-Assignment

- $X = 1$; **while** ($X < N$) { **if** (P) $X = X + 1$; }

```

      X = 1
H:   if  $X \geq N$  goto E
      if  $\neg P$  goto H
      X = X + 1
      goto H
E:   ....

```



```

       $X_1 = 1$ 
H:    $X_2 = \phi(X_1, X_4)$ 
      if  $X_2 \geq N$  goto E
      if  $\neg P$  goto L
       $X_3 = X_2 + 1$ 
L:    $X_4 = \phi(X_2, X_3)$ 
      goto H
E:   ....

```

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```



```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\gamma$  (flag, x1, x2);
y = x3 * a
```

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Use recursive γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

if (flag_1) { $x_1 = 1$; }

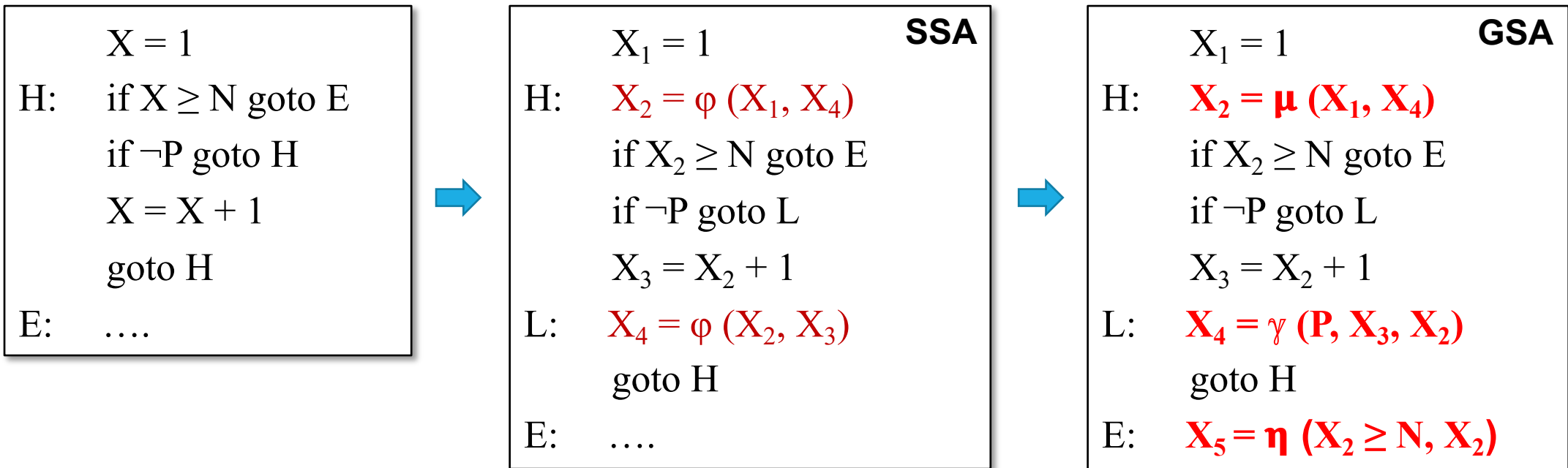
else if (flag_2) { $x_2 = 2$; }

else { $x_3 = 3$; }

$x_4 = \gamma (\text{flag}_1, x_1, \gamma (\text{flag}_2, x_2, x_3));$

Gated SSA

- **Feature 3:** Using μ and η for loop variables
- $X = 1$; **while** ($X < N$) { **if** (P) $X = X + 1$; }



SSA Complexity

- Intuitively, code in SSA is much larger than the non-SSA version
- $X = 1$; **while** ($X < N$) { **if** (P) $X = X + 1$; }

```

    X = 1
H:   if X ≥ N goto E
      if ¬P goto H
      X = X + 1
      goto H
E:   ....

```



```

    X1 = 1
H:   X2 = φ (X1, X4)
      if X2 ≥ N goto E
      if ¬P goto L
      X3 = X2 + 1
L:   X4 = φ (X2, X3)
      goto H
E:   ....

```

SSA Complexity

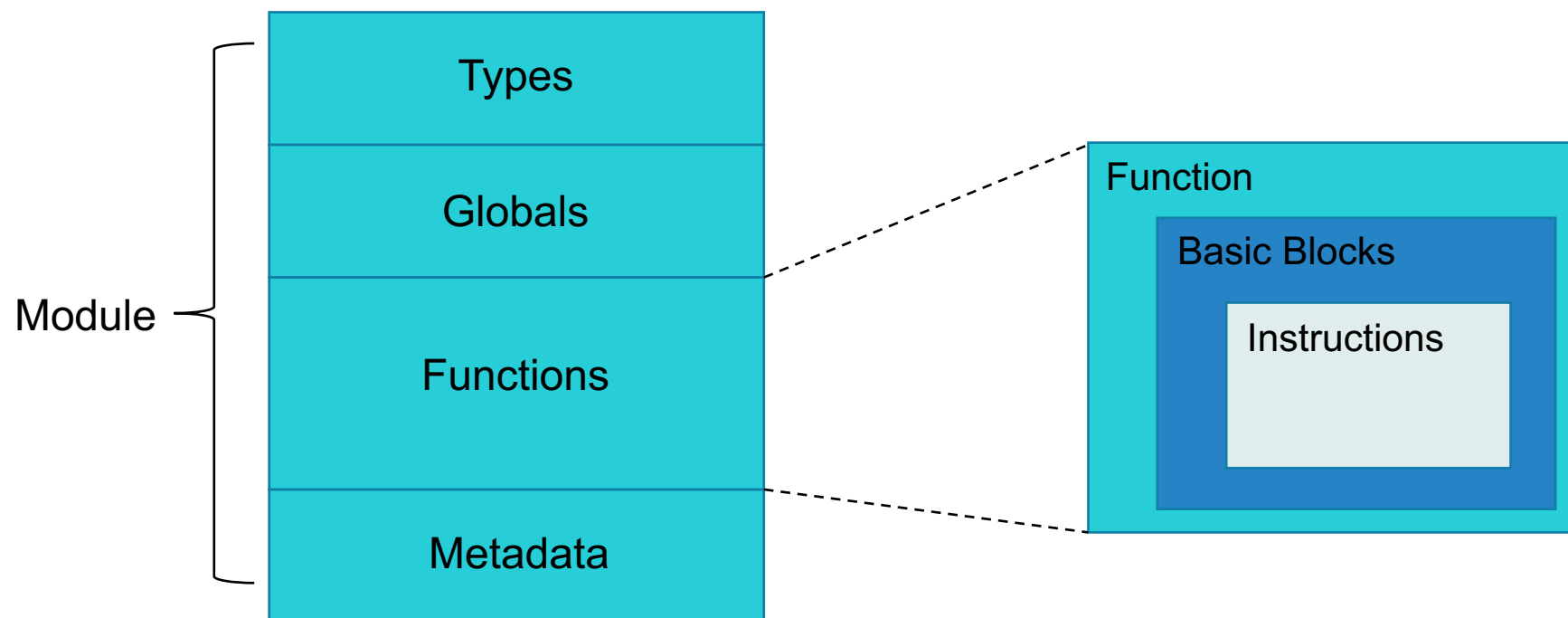
- Space
 - Common three-address code vs. SSA code
 - **Almost Linear!**
- Time
 - From common three-address code to SSA code
 - **Almost Linear!**
- **How? What's the Algorithm?**

PART II: LLVM IR

LLVM IR / Bitcode

<https://llvm.org/docs/LangRef.html>

- `clang -c -emit-llvm hello.c [-o hello.bc]/[-S -o hello.ll]`
- `llvm-dis hello.bc -o hello.ll` `llvm-as hello.ll -o hello.bc`



LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {  
    if (val == 0)  
        return 1;  
    return val * factorial(val - 1);  
}
```

```
define i32 @factorial(i32 %0) {  
1:  
    %2 = icmp eq i32 %0, 0  
    br i1 %2, label %7, label %3  
  
3:  
    %4 = add i32 %0, -1  
    %5 = call i32 @factorial(i32 %4)  
    %6 = mul i32 %5, %0  
    br label %7  
  
7:  
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]  
    ret i32 %8  
}
```

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
    return val * factorial(val - 1);
}
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
    br i1 %2, label %7, label %3, !dbg !16

3:
    %4 = add i32 %0, -1, !dbg !17
    %5 = call i32 @factorial(i32 %4), !dbg !18
    %6 = mul i32 %5, %0, !dbg !19
    br label %7, !dbg !20

7:
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ], !dbg !13
    ret i32 %8, !dbg !21
}
```

```
clang -g -c -emit-llvm hello.c -S -o hello.ll
```

LLVM Types

<https://llvm.org/docs/LangRef.html>

- Void Type
 - void
- Primitive Types
 - i32, i8, i1, ...
 - float, double, ...
- Function Type
 - return-type (parameter types)
 - i32 (i32)
- Pointer Type
 - i32*, [40 x i32]*
 - ptr
- Array Type
 - [40 x i32]
 - [30 x i32*]
- Structure Type
 - { i32, [3 x i32], i8 }
 - ...

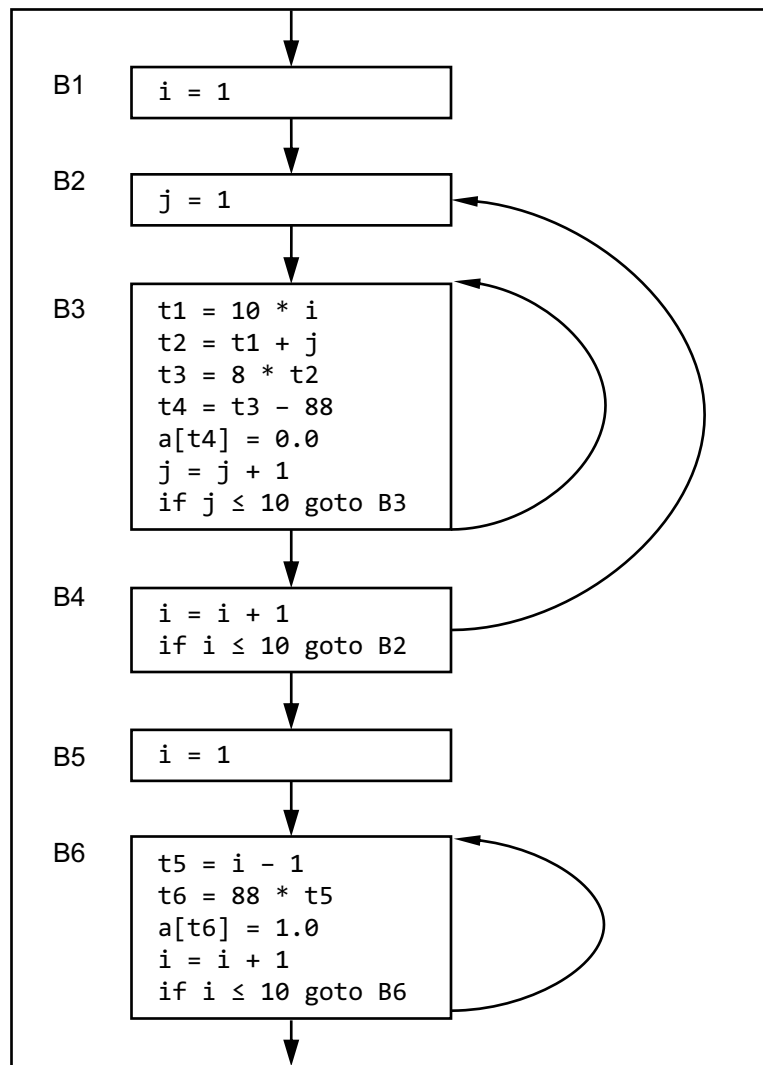
PART III: Translation into SSA

Basic Block

- Partition three-address code to basic blocks
 - A basic block is a sequence of consecutive instructions
 - The first instruction of each basic block is a leader
 - **[1]** The first instruction of a function is a leader
 - **[2]** Any instruction that is the target of a jump is a leader
 - **[3]** Any instruction that follows a jump is a leader

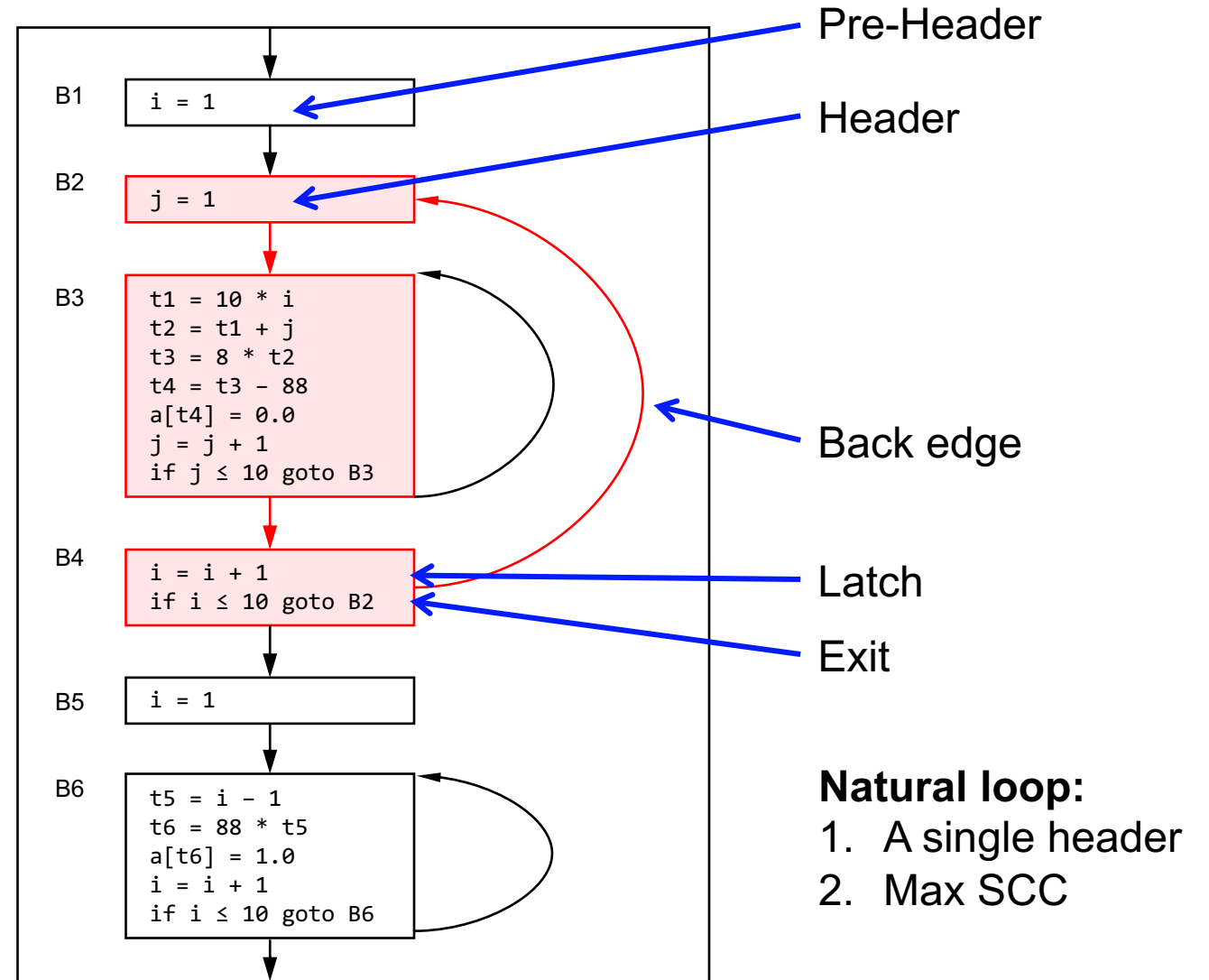
Control Flow Graph

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



Dominance

- What does that mean for a single header?
- The header dominates all blocks in the loop.
 - A **dom** B
 - if all paths from Entry to B goes through A



Post-Dominance

- **A dom B**
 - if all paths from Entry to B goes through A
- **A post-dom B**
 - if all paths from B to Exit goes through A
- **Strict** (post-)dominance – A (post-)dom B but $A \neq B$
- **Immediate** dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B

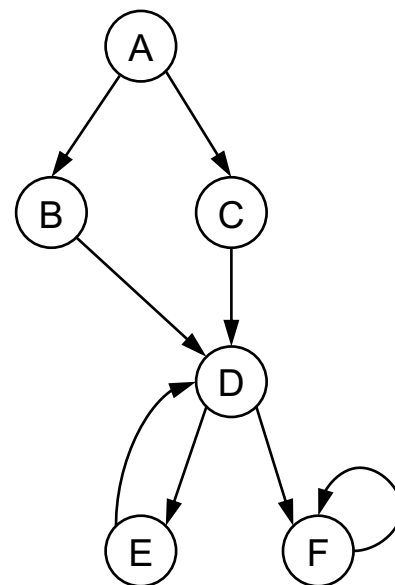
Dominator Tree

- Almost linear time to build a dominator tree.

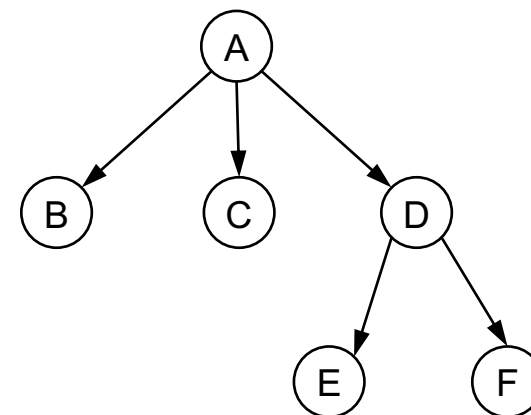
- Node: Block
- Edge: Immediate dom relation

- (Nearest) common dominator

- LCA – lowest common ancestor
- Almost linear preprocessing time
- Constant query time
- **Application:** placing variable definition



Flow Graph



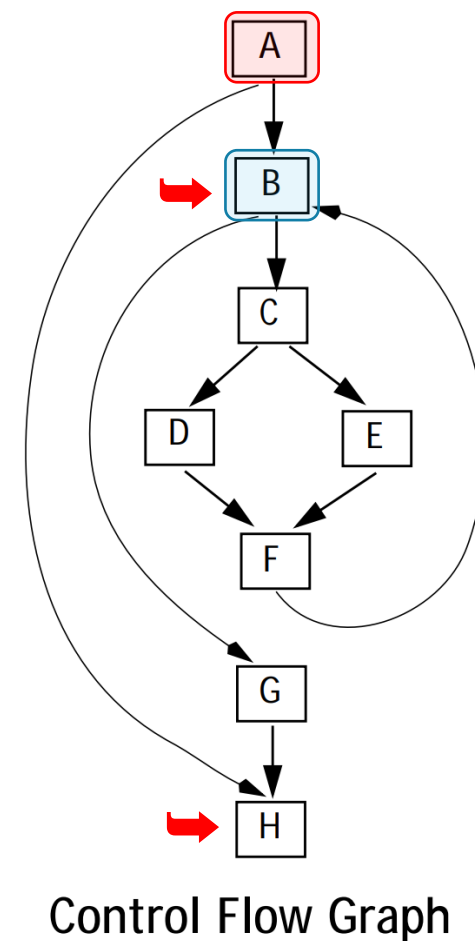
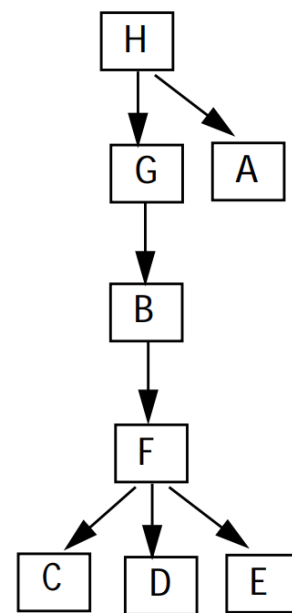
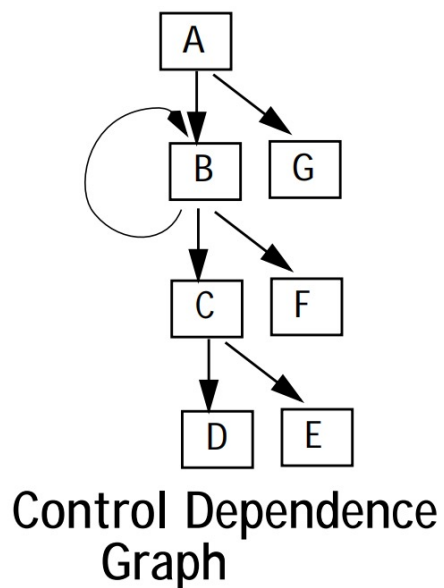
Dominator Tree

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - The execution result of A determines if B will be executed
- Block B is control dependent on Block A if and only if
 - A has multiple successors
 - Not all successors can reach B
- By definition, we always need a forward traversal from all successors of B to test if A ctrl-depends on B. **Too expensive!!**

Dominance vs. Ctrl Dependence

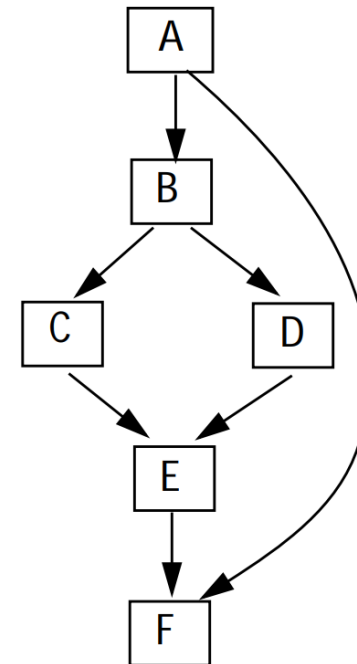
- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



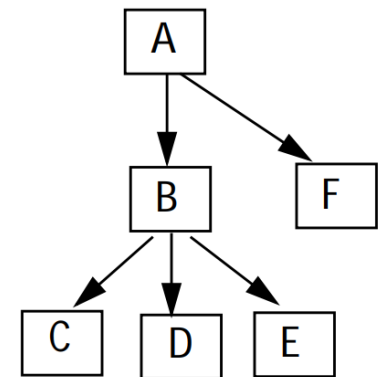
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	{ }



Control Flow Graph



Dominator Tree

Dominance Frontier

- Extend the definition of DF to a set of blocks
 - $DF(\mathbf{Bset}) = \bigcup_{B \in \mathbf{Bset}} DF(B)$

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset

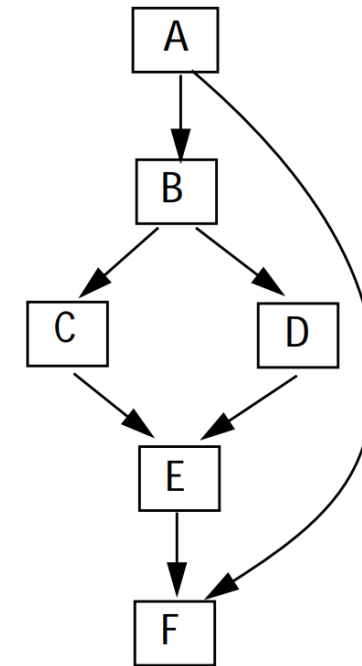
Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset);$

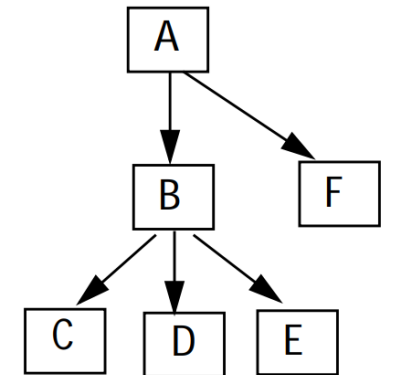
Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(\text{Bset}); \text{Bset} = \text{Bset} \cup DF_1$
 - $DF_2 = DF(\text{Bset}); \text{Bset} = \text{Bset} \cup DF_2$
 -
 - until fixed point! (i.e., $DF_n = DF_{n-1}$)
- $IDF(\{A, B, C\}) = \{ ? \}$

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	{ }



Control Flow Graph



Dominator Tree

IDF vs. SSA

- Where should we insert ϕ to translate IR to SSA?
- By definition, merge point of multiple paths for a variable
- **The merge point of multiple definitions of a variable**

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```

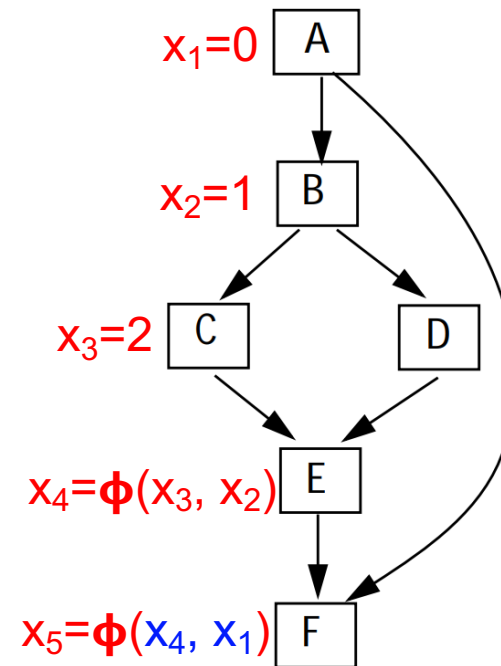
```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\phi$  (x1, x2);
y = x3 * a
```

IDF vs. SSA

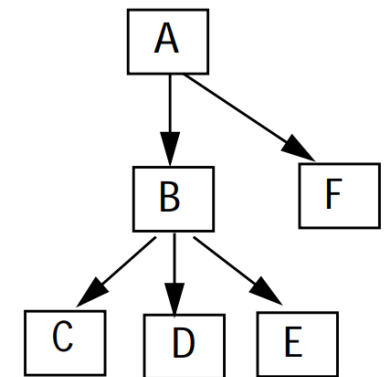
- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}

- find proper definitions by climbing DT



Control Flow Graph



Dominator Tree